Part 1: Understanding Consensus

```
====================
SCENARIO: HAPPY PATH
====================

====================
Proposal Value = instances=10
====================
Acceptors: 3
Required Quorum: 2

A1 PROMISE
A2 PROMISE
A3 PROMISE
Promises Received: 3/3
A1 ACCEPTED
A2 ACCEPTED
A3 ACCEPTED
Acceptances Received: 3/3
VALUE CHOSEN: instances=10
L1 LEARNED CHOSEN VALUE: instances=10
```

The proposer initiates a consensus process, proposes `instances=10`, and sends a `Prepare` request to the acceptors; when at least two acceptors accept the same proposal, the required quorum is reached, and that value is selected. The acceptors first respond to the `Prepare` request by making a `Promise`, and then accept the proposal that complies with the protocol rules; the learners receive and record the final selected value.

Part 2: Acceptor Failure

```
====================
SCENARIO: ACCEPTOR FAILURE
====================

A3 has failed.

====================
Proposal Value = instances=10
====================
Acceptors: 3
Required Quorum: 2

A1 PROMISE
A2 PROMISE
A3 OFFLINE
Promises Received: 2/3
A1 ACCEPTED
A2 ACCEPTED
A3 OFFLINE
Acceptances Received: 2/3
VALUE CHOSEN: instances=10
L1 LEARNED CHOSEN VALUE: instances=10
```

Compared to the `3/3` response in the happy-path scenario, following the A3 failure in this instance, only A1 and A2 responded; however, since the quorum of 2 was still met, `instances=10` was successfully selected. This deployment can tolerate at most one receiver failure; if another failure occurs before consensus is reached, a majority cannot be formed, and the consensus process will halt. Paxos assumes that nodes may fail but will not act maliciously, and that they reliably preserve the protocol state; its fault tolerance relies on majority agreement, rather than unanimous consent.

Part 3: Competing Proposers

```
E:\UoB_all\System_S3c\week5\paxos\paxos-lab>npm run proposers

> paxos-lab@1.0.0 proposers
> node scenarios/competing_proposers.js


P1 -> Prepare(101)

P2 -> Prepare(102)

P1 -> Accept(101)
A1 REJECTED proposal 101
A1 REJECTED
A2 REJECTED proposal 101
A2 REJECTED
A3 REJECTED proposal 101
A3 REJECTED

P2 -> Accept(102)
A1 ACCEPTED
A2 ACCEPTED
A3 ACCEPTED
```

In this competition, P2 was successful, and the winning proposal was number 102. After receiving `Prepare(102)`, the receiver commits to no longer accepting proposals with lower numbers; therefore, it rejects P1's `Accept(101)` and accepts P2's `Accept(102)`. Proposal numbers are used to establish order, distinguish between new and old requests, and coordinate competition; a higher number does not indicate that a decision is more correct or optimal—it merely determines which proposal can proceed. Without numbers, the receiver would be unable to process conflicting requests according to these rules, which could compromise the security of the consensus.

Part 4: Partial Acceptance

```
> paxos-lab@1.0.0 partial
> node scenarios/partial_acceptance.js


====================
SCENARIO: PARTIAL ACCEPTANCE
====================

A1 reports:
(101, instances=10)

P2 wants:
instances=20

====================
Proposal Value = instances=20
====================
Acceptors: 3
Required Quorum: 2

A2 PROMISE
A3 PROMISE
Promises Received: 2/3
A1 ACCEPTED
A2 ACCEPTED
A3 ACCEPTED
Acceptances Received: 3/3
VALUE CHOSEN: instances=20
L1 LEARNED CHOSEN VALUE: instances=20
```

In this round, P2 did not adopt the previous value but ultimately selected `instances=20`: Although A1 had previously accepted `(101, instances=10)`, the Promises in this round came only from A2 and A3, and no previous values were reported; therefore, P2 could continue to propose new values. Only when the collected Promises include an accepted proposal must P2 adopt the highest-numbered value among them. Acceptors must retain the highest Promise number, as well as the numbers and values of accepted proposals; forgetting this history may lead to broken promises or the election of conflicting values. Paxos ensures, through the quorum intersection and historical value retention rules, that a value already selected by a majority will not be overwritten; since the value accepted only by A1 has not yet been selected, the outcome of this round does not violate safety.

Part 5: Preserving Chosen Values

```
====================
SCENARIO: PRESERVING A CHOSEN VALUE
====================

Previously chosen value:
(90, instances=12)

New proposer wants:
instances=20

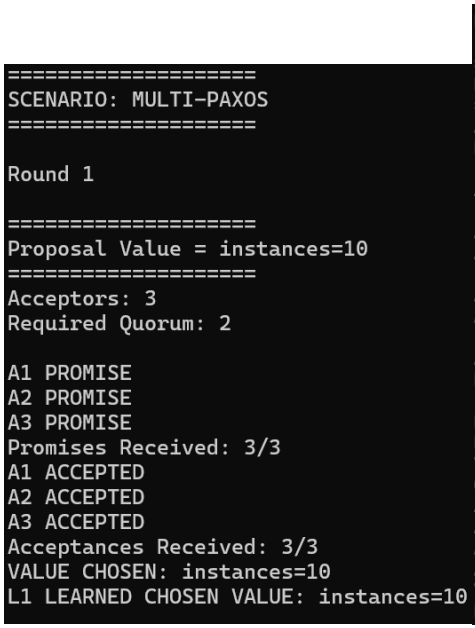
====================
Proposal Value = instances=20
====================
Acceptors: 3
Required Quorum: 2

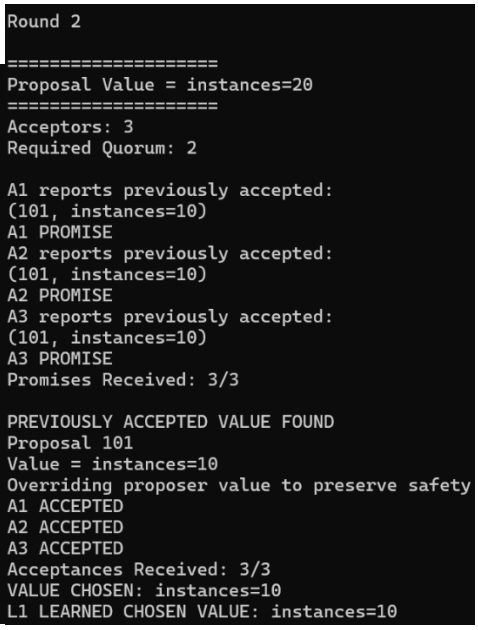
A1 reports previously accepted:
(90, instances=12)
A1 PROMISE
A2 reports previously accepted:
(90, instances=12)
A2 PROMISE
A3 reports previously accepted:
(90, instances=12)
A3 PROMISE
Promises Received: 3/3

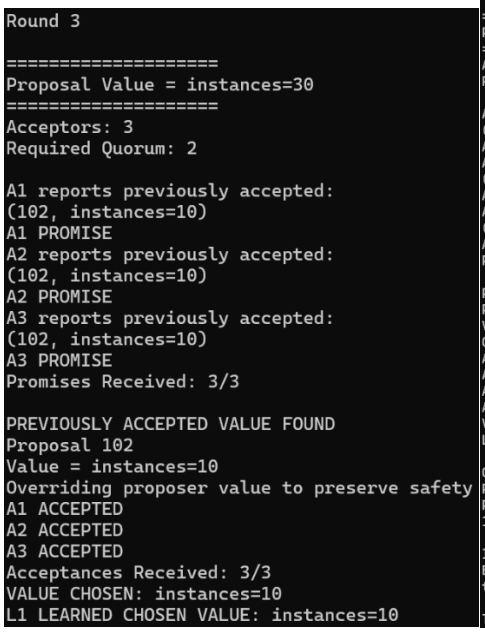
PREVIOUSLY ACCEPTED VALUE FOUND
Proposal 90
Value = instances=12
Overriding proposer value to preserve safety
A1 ACCEPTED
A2 ACCEPTED
A3 ACCEPTED
Acceptances Received: 3/3
VALUE CHOSEN: instances=12
L1 LEARNED CHOSEN VALUE: instances=12
```

The previously selected value was `instances=12` from Proposal 90, and the new proposer wishes to propose `instances=20`. However, the receiver reported the old proposal in the Promise, and the new proposer must adopt the highest-numbered accepted value in the response; therefore, `instances=12` is ultimately selected again. Any two majority quorums must have an intersection, allowing subsequent proposers to detect and preserve existing decisions. This preserves Paxos's safety: the same consensus instance cannot select two different values. If selected values could be overridden, different nodes might execute operations based on different decisions, compromising system consistency.

Part 6: Multi-Paxos and Stable Proposers



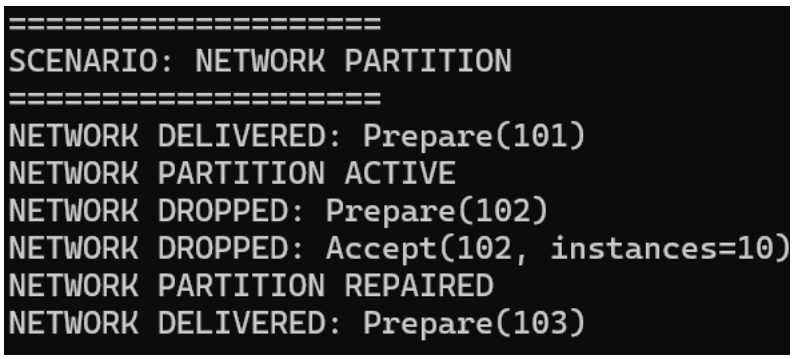






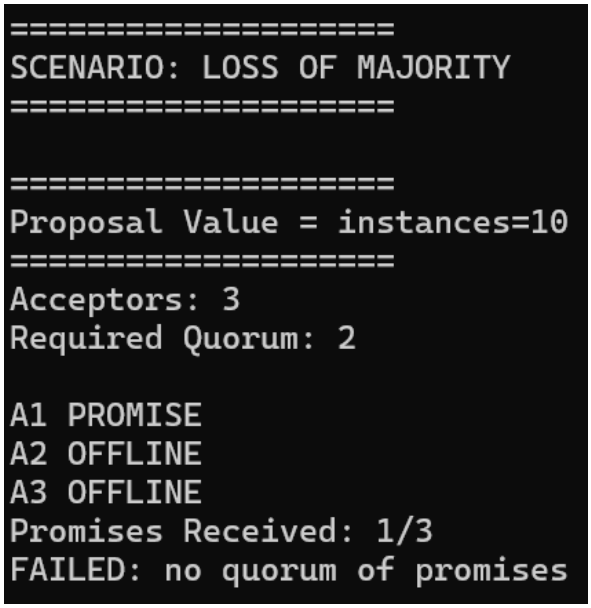
All four rounds were coordinated by P1, with proposal numbers in the sequence 101 → 102 → 103 → 104. Although the requested value increased from `instances=10` to `instances=40`, subsequent rounds must preserve the existing decision; therefore, `instances=10` was ultimately selected in all rounds. Basic Paxos itself does not specify an explicit leader election; when a proposer is consistently successful and competition diminishes, it can become a stable coordinator, but this does not necessarily occur. Multi-Paxos is designed for consensus on a sequence of values, and under appropriate conditions, a stable leader can eliminate the need for repeated `Prepare` operations on subsequent log slots, thereby reducing communication overhead.

Part 7.1: Breaking Paxos Assumptions



Before the partition, `Prepare(101)` was delivered normally; during the partition, `Prepare(102)` and `Accept(102)` were discarded; after recovery, `Prepare(103)` was successfully delivered. A network partition primarily affects **liveness—that is, whether the protocol can continue to progress**—but does not necessarily compromise safety: as long as the quorum and state preservation rules are followed, conflicting values will not be elected. If one side can still reach a majority of acceptors, consensus may still proceed; the side unable to form a majority must wait. Paxos halts decision-making in this scenario to prevent different partitions from independently selecting conflicting values, thereby temporarily sacrificing availability to protect consistency. This output only shows message delivery status and does not directly indicate whether consensus has been reached.

Part 7.2: Majority Loss



During this run, A2 and A3 were offline, and only A1 returned a Promise. The response count was `1/3`, which did not meet the quorum of 2; therefore, the protocol halted at the Prepare phase and could not proceed to the Accept phase. At this point, the majority availability requirement was no longer met, and Paxos could not continue. Safety was not compromised because the system did not elect a conflicting value; if a majority of nodes remain unrecoverable, consensus cannot be reached. This demonstrates that Paxos protects consistency by halting the process and cannot make a decision in the absence of a majority.

Part 7.3: State Loss

```
==================
SCENARIO: STATE LOSS
==================

Before Loss:
Acceptor {
  id: 'A1',
  highestPromised: 101,
  acceptedProposal: { number: 101, value: 'instances=10' },
  online: true
}

After Loss:
Acceptor {
  id: 'A1',
  highestPromised: 0,
  acceptedProposal: null,
  online: true
}

WARNING:
Acceptor forgot previous history.
Paxos safety assumption violated.
```

The receiver must retain the highest promise number, as well as the numbers and values of accepted proposals. In this A1 event, `highestPromised` changes from 101 to 0, and `acceptedProposal` changes from `(101, instances=10)` to `null`, indicating that the historical state has been lost. Persistent storage ensures that nodes continue to honor their original commitments after a failure or restart and report historical values to subsequent proposers. If multiple acceptors forget the history, they may accept conflicting proposals, leading to the election of different values within the same consensus instance. This violates Paxos's assumption of reliable, persistent storage of the protocol state; however, the current output only demonstrates state loss and does not yet directly show conflicting decisions.

Part 7.4: Quorum Misconfiguration

```
====================
SCENARIO: QUORUM MISCONFIGURATION
====================

Correct Majority:
3

Modified Majority:
2

Possible Quorums:
Value A => A1 A2
Value B => A4 A5

WARNING:
Quorum intersection removed.
Different values could be chosen.
```

This configuration reduces the quorum required for the five recipients from 3 to 2, thereby losing the property that any two quorums must have an intersection. For example, A1 and A2 might support value A, while A4 and A5 might support value B. Since the two groups have no common nodes, they cannot constrain each other based on historical states, and thus may elect two conflicting values. Overlapping quorums allow previous decisions to be carried forward to subsequent proposals and form the foundation of Paxos's safety. This

demonstrates that protocol configuration is also part of correctness: even if the code follows the process, incorrect quorum parameters will still undermine safety guarantees. This output illustrates this risk without actually executing a conflicting consensus.

Part 8: Hardening a Paxos Deployment

This section will discuss some mitigation measures for dealing with failures, then attempt to test them and discuss the results.

- Protect Quorum Rules

```
========== 1. Quorum ==========
```

(index)	Cause	Result	Last ID	New
0	'Q=1 < 2'	'BLOCKED'	'-'	'N'

```
P1 -> Prepare: -; P1 -> Accept: -

Quorum: 1 -> 2 | Retry

Online: 3/3

P1 -> Prepare(101) | Request: instances=10
```

(index)	Node	Prepare	Prior	Accept	Value
0	'A1'	'PROMISE'	'-'	'ACCEPTED'	'instances=10'
1	'A2'	'PROMISE'	'-'	'ACCEPTED'	'instances=10'
2	'A3'	'PROMISE'	'-'	'ACCEPTED'	'instances=10'

(index)	Phase	Count	Quorum	Result
0	'Prepare'	3	2	'Y'
1	'Accept'	3	2	'Y'

```
Chosen: instances=10
```

In this scenario, the system has three recipients, and the safe majority quorum is at least 2. When quorum=1, the two independent recipients may support different values, so the program refuses to run before creating a proposal; after restoring quorum=2, all three recipients return a Promise and accept the proposal, ultimately selecting instances=10.

- Detect State Loss

```
Online: 3/3

P1 -> Prepare(101) | Request: instances=10
```

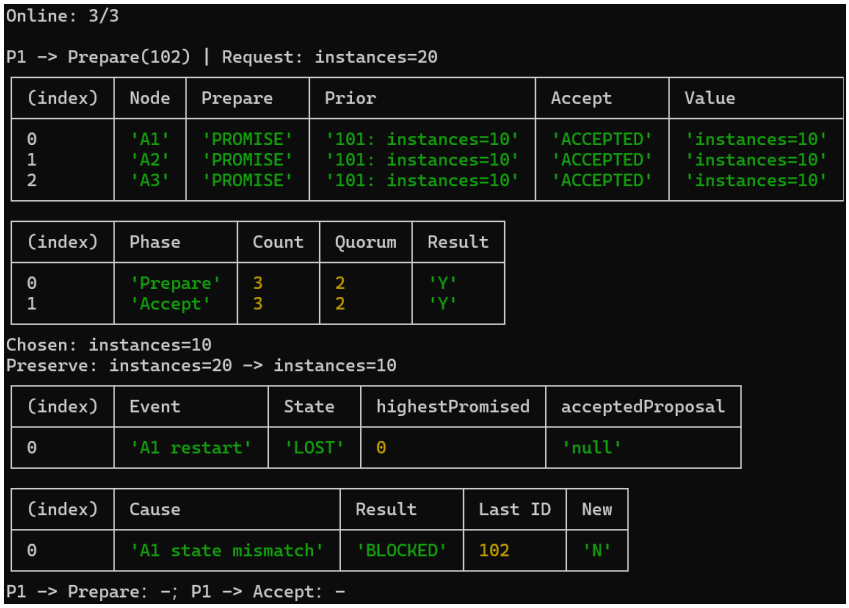
(index)	Node	Prepare	Prior	Accept	Value
0	'A1'	'PROMISE'	'-'	'ACCEPTED'	'instances=10'
1	'A2'	'PROMISE'	'-'	'ACCEPTED'	'instances=10'
2	'A3'	'PROMISE'	'-'	'ACCEPTED'	'instances=10'

(index)	Phase	Count	Quorum	Result
0	'Prepare'	3	2	'Y'
1	'Accept'	3	2	'Y'

```
Chosen: instances=10
```

(index)	Event	State	highestPromised	acceptedProposal
0	'A1 restart'	'RESTORED'	101	'RESTORED'

In the first round of consensus, `instances=10` was selected; subsequently, A1 restarted while preserving its state. In the next round, Proposal 102 attempted to propose `instances=20`, but all three receivers reported `Prior = 101: instances=10`, so the proposal ultimately retained the existing value. This demonstrates that, within the same Paxos consensus instance, a decision that has already been selected cannot be overridden by a subsequent proposal.



On the second restart, the A1 state was intentionally discarded, setting `highestPromised` to 0 and `acceptedProposal` to null; a check revealed that the state was inconsistent with the checkpoint, which blocked subsequent requests, and the number ultimately remained at 102. It is important to note that while this mechanism can detect anomalies and halt operations, it does not recover lost data.

- Protect Against Majority Loss

```
========== 3. Majority ==========

Online: 1/3
```

(index)	Cause	Result	Last ID	New
0	'Online=1 < Q=2'	'BLOCKED'	'-'	'N'

```
P1 -> Prepare: -; P1 -> Accept: -

A2: OFF -> ON | A3: OFF

Online: 2/3

P1 -> Prepare(101) | Request: instances=20
```

(index)	Node	Prepare	Prior	Accept	Value
0	'A1'	'PROMISE'	'-'	'ACCEPTED'	'instances=20'
1	'A2'	'PROMISE'	'-'	'ACCEPTED'	'instances=20'
2	'A3'	'NO'	'-'	'NO'	'-'

(index)	Phase	Count	Quorum	Result
0	'Prepare'	2	2	'Y'
1	'Accept'	2	2	'Y'

```
Chosen: instances=20
```

When only A1 is online, the system cannot achieve a majority, so it stops creating proposals. At this point, security has not been compromised, but the system cannot proceed; after A2 is restored, A1 and A2 respond normally, while A3 shows “NO” for both “Prepare” and “Accept.” There are still two valid responses across the two phases, which exactly meet the quorum requirement, and instances=20 is ultimately selected.

- Detect Proposer Instability

```
========== 4. Rate Limit ==========
Limit: 3/1000ms | t(ms): 0, 100, 200, 300, 1000

Online: 3/3

P1 -> Prepare(101) | Request: instances=10
```

(index)	Node	Prepare	Prior	Accept	Value
0	'A1'	'PROMISE'	'-'	'ACCEPTED'	'instances=10'
1	'A2'	'PROMISE'	'-'	'ACCEPTED'	'instances=10'
2	'A3'	'PROMISE'	'-'	'ACCEPTED'	'instances=10'

(index)	Phase	Count	Quorum	Result
0	'Prepare'	3	2	'Y'
1	'Accept'	3	2	'Y'

```
Chosen: instances=10

Online: 3/3

P1 -> Prepare(102) | Request: instances=20
```

(index)	Node	Prepare	Prior	Accept	Value
0	'A1'	'PROMISE'	'101: instances=10'	'ACCEPTED'	'instances=10'
1	'A2'	'PROMISE'	'101: instances=10'	'ACCEPTED'	'instances=10'
2	'A3'	'PROMISE'	'101: instances=10'	'ACCEPTED'	'instances=10'

(index)	Phase	Count	Quorum	Result
0	'Prepare'	3	2	'Y'
1	'Accept'	3	2	'Y'

```
Chosen: instances=10
Preserve: instances=20 -> instances=10
```

```
P1 -> Prepare(103) | Request: instances=30
```

(index)	Node	Prepare	Prior	Accept	Value
0	'A1'	'PROMISE'	'102: instances=10'	'ACCEPTED'	'instances=10'
1	'A2'	'PROMISE'	'102: instances=10'	'ACCEPTED'	'instances=10'
2	'A3'	'PROMISE'	'102: instances=10'	'ACCEPTED'	'instances=10'

(index)	Phase	Count	Quorum	Result
0	'Prepare'	3	2	'Y'
1	'Accept'	3	2	'Y'

```
Chosen: instances=10
Preserve: instances=30 -> instances=10

Online: 3/3
```

(index)	Cause	Result	Last ID	New
0	'3/1000ms'	'BLOCKED'	103	'N'

```
P1 -> Prepare: -; P1 -> Accept: -

t=1000ms | Oldest expired -> Retry

Online: 3/3

P1 -> Prepare(104) | Request: instances=40
```

(index)	Node	Prepare	Prior	Accept	Value
0	'A1'	'PROMISE'	'103: instances=10'	'ACCEPTED'	'instances=10'
1	'A2'	'PROMISE'	'103: instances=10'	'ACCEPTED'	'instances=10'
2	'A3'	'PROMISE'	'103: instances=10'	'ACCEPTED'	'instances=10'

(index)	Phase	Count	Quorum	Result
0	'Prepare'	3	2	'Y'
1	'Accept'	3	2	'Y'

```
Chosen: instances=10
Preserve: instances=40 -> instances=10
```

In this test, the system allows three proposals to be created every 1,000 milliseconds. The first three requests generated proposal numbers 101–103; the fourth request was blocked due to exceeding the limit, so the proposal number did not increase, and neither the `Prepare` nor the `Accept` operations were executed. After 1,000 milliseconds had elapsed, the quota was released, and a retry successfully generated proposal 104, indicating that the rate-limiting and recovery mechanisms were functioning as intended.

- Improve Observability

```
========== 5. Audit ==========

Online: 3/3

P1 -> Prepare(101) | Request: instances=20
```

(index)	Node	Prepare	Prior	Accept	Value
0	'A1'	'PROMISE'	'-'	'ACCEPTED'	'instances=20'
1	'A2'	'PROMISE'	'-'	'ACCEPTED'	'instances=20'
2	'A3'	'PROMISE'	'-'	'ACCEPTED'	'instances=20'

(index)	Phase	Count	Quorum	Result
0	'Prepare'	3	2	'Y'
1	'Accept'	3	2	'Y'

```
Chosen: instances=20

Audit:
```

(index)	Field	Data
0	'Proposal'	'101'
1	'Accepted'	'A1=instances=20; A2=instances=20; A3=instances=20'
2	'Promises'	'A1=PROMISE; A2=PROMISE; A3=PROMISE'
3	'Quorums'	'Prepare: 3/2; Accept: 3/2'
4	'Chosen'	'instances=20'

```
PASS: 5/5 test groups
Legend: Y=yes | N=no | -=none/skipped | NO=no response/acceptance
Scope: local rate limit; in-memory state/logs; simulated restart.
```

The log contains entries such as:
“Proposal 101 created,”
“All recipients have made commitments,”
“All recipients have accepted instances=20,”
“Majority was reached in both the Prepare and Accept phases,”
and “The final decision is instances=20.”
These entries link the request, node responses, and the final decision, helping to pinpoint the cause of the failure.